

Table of content

1.0 Query.....	1
1.1 Query 1:Identification of Ponds with Unhealthy Fish in Aquafarm.....	1
1.1.1 Explanation of Query.....	1
1.1.2 Sql Query.....	1
1.1.3 Sample Output.....	1
1.2 Query 2: Equipment Usage Analysis in Aquafarm Operations.....	2
1.2.1 Explanation of Query.....	2
1.2.2 Sql Query.....	2
1.2.3 Sample Output.....	3
1.3 Query 3: Total spent by the customers purchasing from the aquafarm.....	3
1.3.1 Explanation of Query.....	3
1.3.2 Sql Query.....	4
1.3.3 Sample Output.....	4
1.4 Query 4: Water Quality Summary per Batch.....	5
1.4.1 Explanation of Query.....	5
1.4.3 Sample Output.....	6
1.5 Query 5: Total Operational Cost per Batch.....	6
1.5.1 Explanation of Query.....	6
1.5.2 Sql Query.....	7
1.5.3 Sample Output.....	7
1.6 Query 6: Inventory Turnover and Reorder Alert.....	8
1.6.1 Explanation of Query.....	8
1.6.2 Sql Query.....	8
1.6.3 Sample Output.....	9

1.0 Query

1.1 Query 1: Identification of Ponds with Unhealthy Fish in Aquafarm

1.1.1 Explanation of Query

This query is used to identify ponds that currently contain unhealthy fish in Aquafarm operations. It retrieves the pond ID and location from the Pond table and links it to the Batch table to determine which fish batches are being raised in each pond. The query then joins with the Health_Status table to obtain the health condition of each batch. By applying a WHERE clause, it filters only the records where the health condition indicates potential issues, such as infection, poor water quality, or stress. This allows management to quickly detect which ponds require attention or intervention. In real-life scenarios, this analysis is important for early problem detection, improving fish survival rates, and ensuring proper farm management by enabling timely treatment and monitoring of affected ponds.

1.1.2 Sql Query

```
SELECT p.pond_id, p.pond_loc, hs.health_condition
FROM Pond p
JOIN Batch b ON p.pond_id = b.pond_id
JOIN Health_Status hs ON b.Health_id = hs.Health_id
WHERE hs.health_condition IN ('Infection', 'Poor Quality', 'Mild
Stress');
```

1.1.3 Sample Output

pond_id	pond_loc	health_condition
102	Zone-A2	Mild Stress
103	Zone-B1	Infection
105	Zone-C1	Poor Quality

This output indicates which ponds currently contain unhealthy fish in the Aquafarm operations. Firstly, the 'pond_id' column list is the unique identifier for each pond. The next column is to show the physical location of the pond is situated and named as 'pond_loc'.

While the last column is to view the 'health_condition' of the specific health issue that is affecting the fish in that pond. As seen from the result, pond 102 in Zone-A2 has a health condition of mild stress meaning that the fishes in that pond are experiencing minor health issues and it is required for monitoring. For the second row of the result, infection has occurred at Pond 103 of Zone-B1. This means that the pond has the most serious condition that needs immediate treatment. Lastly, Pond 105 in Zone-C1 is viewed as 'Poor Quality' to indicate that the water quality of the pond was not optimal. Hence, the management can quickly find out which ponds are requiring maintenance and prevent the health issue from spreading to other ponds when the results are filtered to show only the unhealthy condition.

1.2 Query 2: Equipment Usage Analysis in Aquafarm Operations

1.2.1 Explanation of Query

This query is used to determine how frequently each piece of equipment is utilized in Aquafarm operations. It retrieves the equipment name from the Equipment table and counts how many times each item appears in the Equipment_Usage records, which represent actual usage activities. By joining the Equipment_Usage table with the Equipment table using the equipment ID, the query links usage data with the corresponding equipment details. The COUNT() function calculates the total number of times each equipment is used, while the GROUP BY clause groups the results by equipment name so that each item appears only once. Finally, the ORDER BY clause sorts the results in descending order based on usage count, allowing management to compare equipment usage levels and identify the most frequently used equipment. This is useful in real-life Aquafarm scenarios for maintenance planning, efficient resource allocation, and ensuring that heavily used equipment is properly maintained to support continuous farm operations.

1.2.2 Sql Query

```
SELECT e.eq_name, COUNT(*) AS usage_count
FROM Equipment_Usage eu
JOIN Equipment e ON eu.eq_id = e.eq_id
GROUP BY e.eq_name
ORDER BY usage_count DESC;
```

1.2.3 Sample Output

eq_name	usage_count
Water Pump	3
Oxygen Aerator	1
Automatic Feeder	1
PH Meter	1
Harvesting Net	1
Water Heater	1

This output shows how often each type of equipment is used in Aquafarm operations. The 'eq_name' column lists the name of each equipment, while the 'usage_count' column indicates the number of times that equipment appears in the Equipment_Usage records. From the result, the **Water Pump** has the highest usage count of 3, meaning it is the most frequently used equipment in daily activities. All other equipment, such as Oxygen Aerator, Automatic Feeder, PH Meter, Harvesting Net, and Water Heater, each have a usage count of 1, indicating they are used less frequently. Because the results are sorted in descending order, the most commonly used equipment appears at the top, making it easy to identify key resources. This helps management understand which equipment is heavily relied on and may require more frequent maintenance or monitoring.

1.3 Query 3: Total spent by the customers purchasing from the aquafarm

1.3.1 Explanation of Query

This query helps to analyze the customer purchasing behavior and to identify which customers contribute the most revenue to the aquafarm business. The customer name and the contact information are obtained from the CUSTOMER table and then uses JOIN to link their sales transaction records from the SALES_TRANSACTION table using the Customer_id as the common key. Then, the query uses the SUM aggregation function to calculate the total quantity of fish purchased and the total amount of money spent by each of the customers. However, the total amount of money spent is calculated by multiplying the total quantity of fish purchased with the sales quantity by price for each transaction. Apart from that, the query also applies the GROUP BY function to group all the transactions by each of the customer to make sure that the aggregation functions are able to calculate the totals.

per customer rather than across all records without causing a problem. Finally, in order to find the total amount spent by the customers purchasing from the aquafarm in descending order, ORDER BY function is used to place the highest value of the total spent at the top of the result set. Therefore, it is easier to identify their most valuable customers in a quicker way and understand the purchasing patterns.

1.3.2 Sql Query

```
SELECT c.Customer_name, c.Customer_contact,
SUM(st.Sales_quantity) AS total_quantity_purchased,
ROUND(SUM(st.Sales_quantity * st.Price), 2) AS total_spent
FROM CUSTOMER c
JOIN SALES_TRANSACTION st ON c.Customer_id = st.Customer_id
GROUP BY c.Customer_id, c.Customer_name, c.Customer_contact
ORDER BY total_spent DESC;
```

1.3.3 Sample Output

Customer_name	Customer_contact	total_quantity_purchased	total_spent
Janice Cheah	011-32143322	50.00	750.00
Ali Ahmad	0123456789	50.00	625.00
Siti Aminah	0172233445	40.00	600.00
Cynthia Ho	0164256781	40.00	600.00
John Tan	0198765432	30.00	375.00

As from this output, the name of each customer is displayed on the column called 'Customer_name' while their contact information is also shown in the 'Customer_contact' column. The next column as 'total_quantity_purchased' is to present the total amount of fish bought by each of the customers across all transactions. Then, it is to view the total revenue generated from the customer in the 'total_spent' column. From this result, Janice Cheah has the highest total spent of 750.00 with a quantity of 50 fishes. So, this made her the most valuable customer with the highest total spent for the aquafarm. Then, Ali Ahmad is at the second place with a total spent of 625.00 by purchasing 50 fishes. Furthermore, both Cynthia Ho and Siti Aminah have spent 600.00 with 40.00 quantity bought and the lowest total spent of 375.00 with quantity of 30 fishes purchased was from John Tan. In this output set, the most valuable customers will be appeared at the top because the results are arranged in a descending order by the total amount spent. As a result, this helps management to understand

what the customer usually buys and uses this information to develop marketing strategies in order to maintain the top customers and increase sales from the customers from the lowest.

1.4 Query 4: Water Quality Summary per Batch

1.4.1 Explanation of Query

This query finds the batches where the environment is affecting the water quality of the pond. It looks at the Water_Quality table to get information. This includes the batch ID, the pond location, the temperature, the pH level and the dissolved oxygen level. The query also gets information from Batch and Pond tables so the management can understand the pond better. The query uses a CASE statement to create an 'alert_status' label. This label shows 'Temperature Alert' if the temperature is too low or too high which falls below 24°C or above 27°C. It also shows 'pH Alert' if pH drops below 6.5 or rises above 8.0. The query only shows 'Within Range' if the temperature and the pH are correct. The WHERE clause filters the results to only show the records that have at least one alert like the temperature or pH out of range. This means it does not show the batches that are completely fine.

1.4.2 Sql Query

```
SELECT
    b.batch_id,
    p.pond_loc,
    wq.temperature,
    wq.pH,
    wq.dissolved_oxygen_lvl,
    CASE
        WHEN wq.temperature < 24 OR wq.temperature > 27 THEN
'Temperature Alert'
        WHEN wq.pH < 6.5 OR wq.pH > 8.0 THEN 'pH Alert'
        ELSE 'Within Range'
    END AS alert_status
FROM Water_Quality wq
JOIN Batch b ON wq.batch_id = b.batch_id
JOIN Pond p ON b.pond_id = p.pond_id
```

```
WHERE (wq.temperature < 24 OR wq.temperature > 27 OR wq.pH < 6.5
OR wq.pH > 8.0)
ORDER BY alert_status, b.batch_id;
```

1.4.3 Sample Output

batch_id	pond_loc	temperature	pH	dissolved_oxygen_lvl	alert_status
1	Zone-A1	25.00	5.2	6.80	pH Alert
4	Zone-B1	27.00	4.8	5.90	pH Alert

The query output shows only batches of pond that have triggered pH alerts or temperature alerts, with pH-value or temperature out of optimal range will be included in the output. For each such pond, it displays the batch id and pond location, its temperature, pH-value and dissolved oxygen level. The column of alert status showing the pond either pH-value or temperature out of optimal range. The alert status column telling staff the problem of water quality of the pond to take immediate action.

1.5 Query 5: Total Operational Cost per Batch

1.5.1 Explanation of Query

This query calculates the total operational cost for each batch by summing three expense categories, feed cost from Feeding_Management, daily activity cost from Daily_Activity_Management, and overhead cost from Cost_Record. It joins the Batch table with Species to display the species name. It then uses LEFT JOIN to link the expense tables. This way it includes batches even if they do not have records in one or more expense categories. The COALESCE function converts any null sums into zero, preventing calculation errors. After grouping by batch and species, the sum function aggregates each cost type, and the three totals are added together to produce total_batch_cost, rounded to two decimal places. Results are ordered from highest to lowest cost, helping management quickly identify the most expensive batches. In real life, the farm manager can use this query to decide whether to continue raising a costly species, negotiate better feed prices or just labour schedules to reduce daily activity expenses.

1.5.2 Sql Query

```
SELECT
    b.batch_id, s.species_name,
    COALESCE(SUM(fm.cost_food), 0) AS feed_cost,
    COALESCE(SUM(dam.cost_actUsed), 0) AS activity_cost,
    COALESCE(SUM(cr.amount), 0) AS overhead_cost,
    ROUND(COALESCE(SUM(fm.cost_food), 0) +
        COALESCE(SUM(dam.cost_actUsed), 0) +
        COALESCE(SUM(cr.amount), 0), 2) AS total_batch_cost
FROM Batch b
JOIN Species s ON b.species_id = s.species_id
LEFT JOIN Batch_Feeding bf ON b.batch_id = bf.batch_id
LEFT JOIN Feeding_Management fm ON bf.feed_id = fm.feed_id
LEFT JOIN Daily_Activity_Management dam ON b.batch_id = dam.batch_id
LEFT JOIN Cost_Record cr ON b.batch_id = cr.batch_id
GROUP BY b.batch_id, s.species_name
ORDER BY total_batch_cost DESC;
```

1.5.3 Sample Output

batch_id	species_name	feed_cost	activity_cost	overhead_cost	total_batch_cost
1	Atlantic Salmon	270.00	90.00	165.00	525.00
2	Blue Nile Tilapia	156.00	45.00	60.00	261.00
5	Channel Catfish	60.00	0.00	40.00	100.00
3	Yellow Perch	38.00	0.00	50.00	88.00
4	Rainbow Trout	48.00	0.00	0.00	48.00
6	Common Carp	40.00	0.00	0.00	40.00

The query output shows six columns each batch. The “batch_id” identifies the batch, and ‘species_name’ shows the fish species raised. The ‘feed_cost’ adds up all money spent on feeding fish from Feeding_Management. The ‘activity_cost’ sums daily operational costs such as water changes, cleaning, and health checks from Daily_Activity_Management. The ‘overhead_cost’ includes expenses like transportation and maintenance from Cost_Record. The ‘total_batch_cost’ column sums these three costs rounded to two decimal places. Batches are listed from highest to lowest total cost. Management can scan the output to identify which batches are most expensive. For example, a batch with high ‘feed_cost’ might mean overfeeding or expensive food. A high ‘activity_cost’ could signal frequent health

interventions. Zeros in any column suggest some information is missing, so someone should double-check records. This breakdown helps management make decisions on feed purchasing, labour allocation, and species profitability.

1.6 Query 6: Inventory Turnover and Reorder Alert

1.6.1 Explanation of Query

This query is designed to monitor inventory remaining and identify the shortage to remind the necessary of reordering before run out of stock and expiring. It evaluates current stock shortage by inserting the predefined thresholds (critical below 20 units, warning below 50 units). It retrieves key information including item details, category, supplier and days until expiry, therefore CASE statements are used to generate an alert-based stock status, and HAVING used to filter the results and show only items requiring attention (critical or warning levels), and orders by quantity in ascending order so the least stock items will appear first on the top of the table output.

1.6.2 Sql Query

```
SELECT
    ii.item_id,
    ii.item_name,
    c.category_name,
    ii.quantity_in_stock,
    s.company_name AS supplier_name,
    CASE
        WHEN ii.quantity_in_stock < 20 THEN 'CRITICAL - Reorder
Immediately'
        WHEN ii.quantity_in_stock < 50 THEN 'Warning - Low Stock'
        ELSE 'Sufficient Stock'
    END AS stock_status,
    DATEDIFF(ii.expiry_date, CURDATE()) AS days_until_expiry
FROM Inventory_Item ii
JOIN Category c ON ii.category_id = c.category_id
JOIN Supplier s ON ii.supplier_id = s.supplier_id
```

```

WHERE ii.expiry_date IS NULL OR ii.expiry_date > CURDATE()
HAVING stock_status IN ('CRITICAL - Reorder Immediately', 'Warning
- Low Stock')
ORDER BY ii.quantity_in_stock ASC;

```

1.6.3 Sample Output

item_id	item_name	category_name	quantity_in_stock	supplier_name	stock_status	days_until_expiry
2	Water Pump	Equipment	10	Aqua Supply	CRITICAL - Reorder Immediately	NULL
5	Water Filter	Equipment	15	Blue Ocean	CRITICAL - Reorder Immediately	NULL
4	Net	Equipment	20	Aqua Supply	Warning - Low Stock	NULL

The query output produces a report which shows only inventory items that are low in stocks and need for reorder attention. For each such inventory item, it displays the item ID and name, its category, current remaining quantity, supplier name, a stock status label either 'CRITICAL - Reorder Immediately' for quantities under 20, or 'Warning - Low Stock' for quantities between 20 and 50, and the number of days remaining before expiry (or NULL if no expiry). Items are listed from the least quantities remaining to the most quantities remaining, meaning the most urgent to reorder items will appear at the top. For example, the first row has item ID 2, 'Water Pump' with quantity 10, stock status 'CRITICAL', supplier 'Aqua Supply', and remaining days until expiry is null since there is no expiry, telling the manager to urgently reorder stock with the Aqua Supply. Items that are considered to have sufficient stock (50 or more) are completely excluded from the output, keeping the report focused only on action needed items.